
```
oAccount.interestRate
```

This would work, for a while. But later the bank might decide on a new policy—say that the interest rate will depend on the amount of money in the account. The interest rate might be calculated like this:

```
def calculateInterestRate(self):
    # Assuming self.balance has been set in another method
    if self.balance < 1000:
        self.interestRate = 1.0
    elif self.balance < 5000:
        self.interestRate = 1.5
    else:
        self.interestRate = 2.0
```

Rather than just relying on a single interest rate value in `self.interestRate`, the `calculateInterestRate()` method determines the current rate based on the account balance.

Any client software that directly accesses `oAccount.interestRate` and uses the value of the instance variable might then get an outdated value, depending on the last time `calculateInterestRate()` was called. And any client software that sets a new `interestRate` may find that the new value is mysteriously changed by some other code that calls `calculateInterestRate()` or when the bank account owner makes a deposit or withdrawal.

If, however, the interest calculation method was named `getInterestRate()` and client software called that instead, the interest rate would always be calculated on the fly and there would be no potential error.

Validating Data

The third reason to avoid direct access when setting a value is that client code can too easily set an instance variable to an invalid value. A better approach is to call a method in the class, whose job is to set the value. As the developer, you can include validation code in that method to ensure that the value being set is appropriate. Consider the code in [Listing 8-3](#), whose purpose is to manage the members of a club.